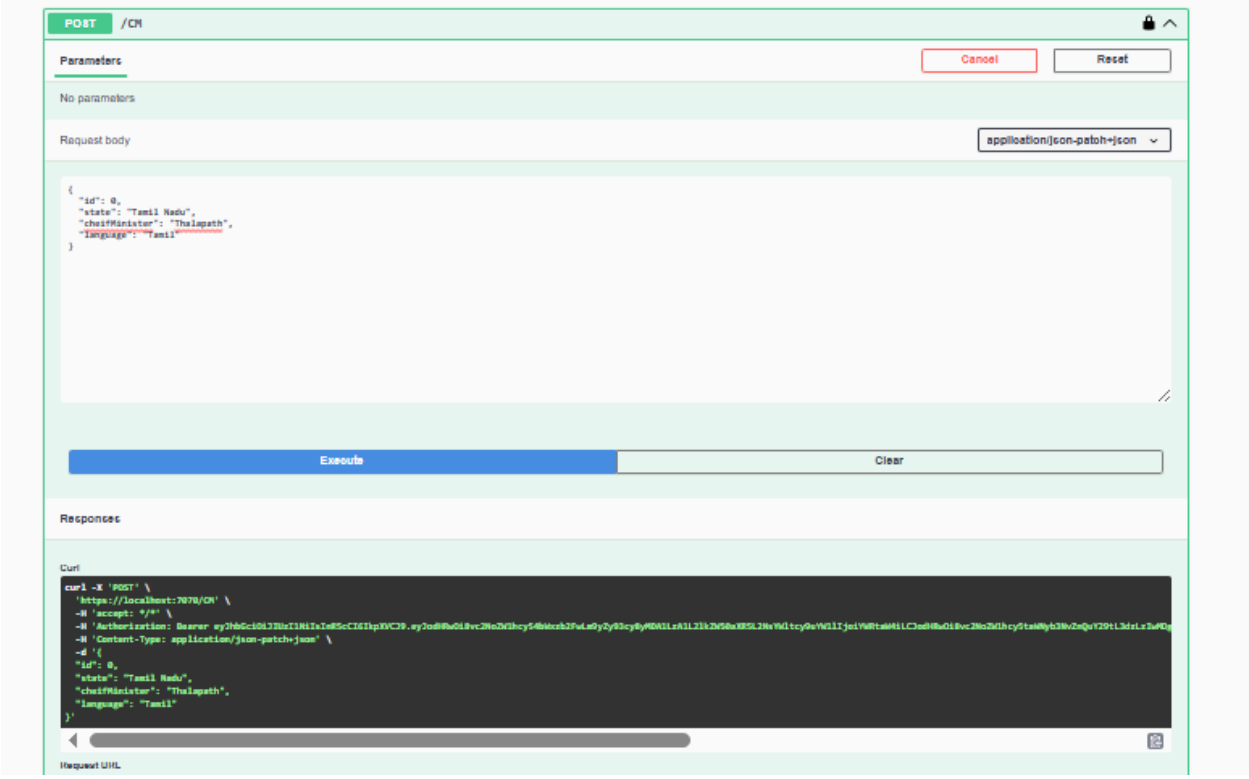
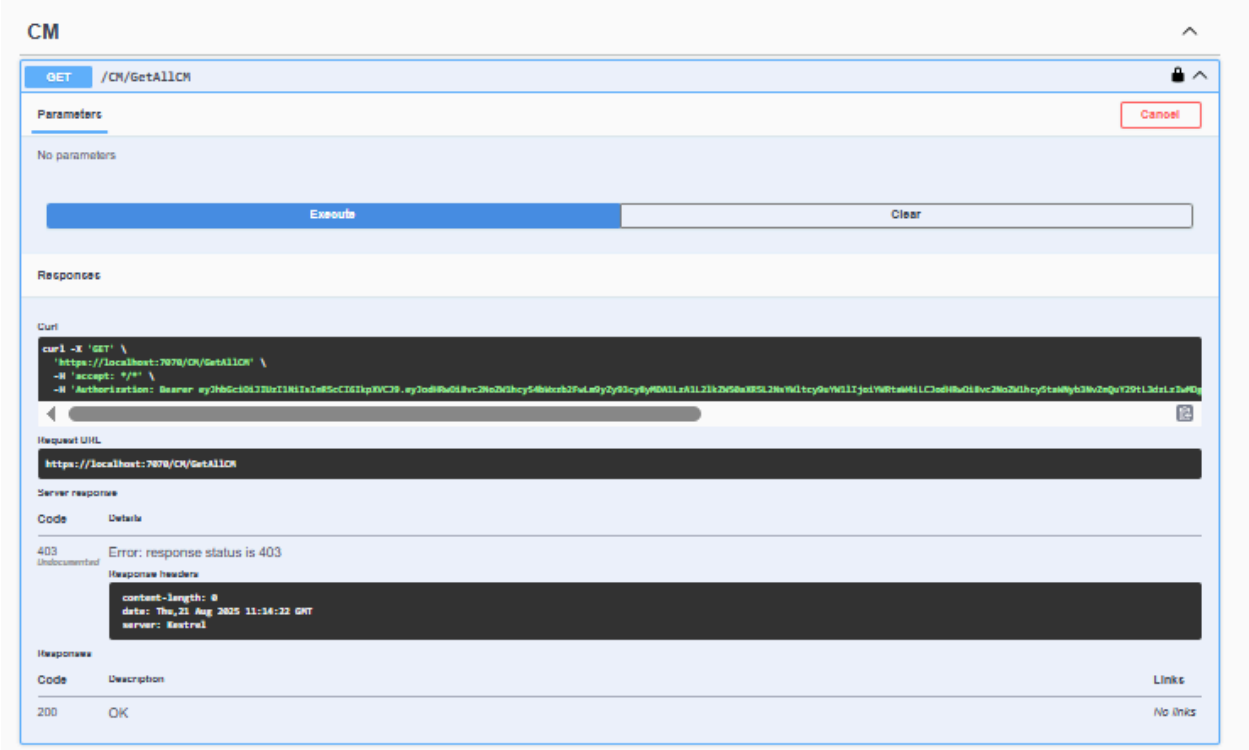
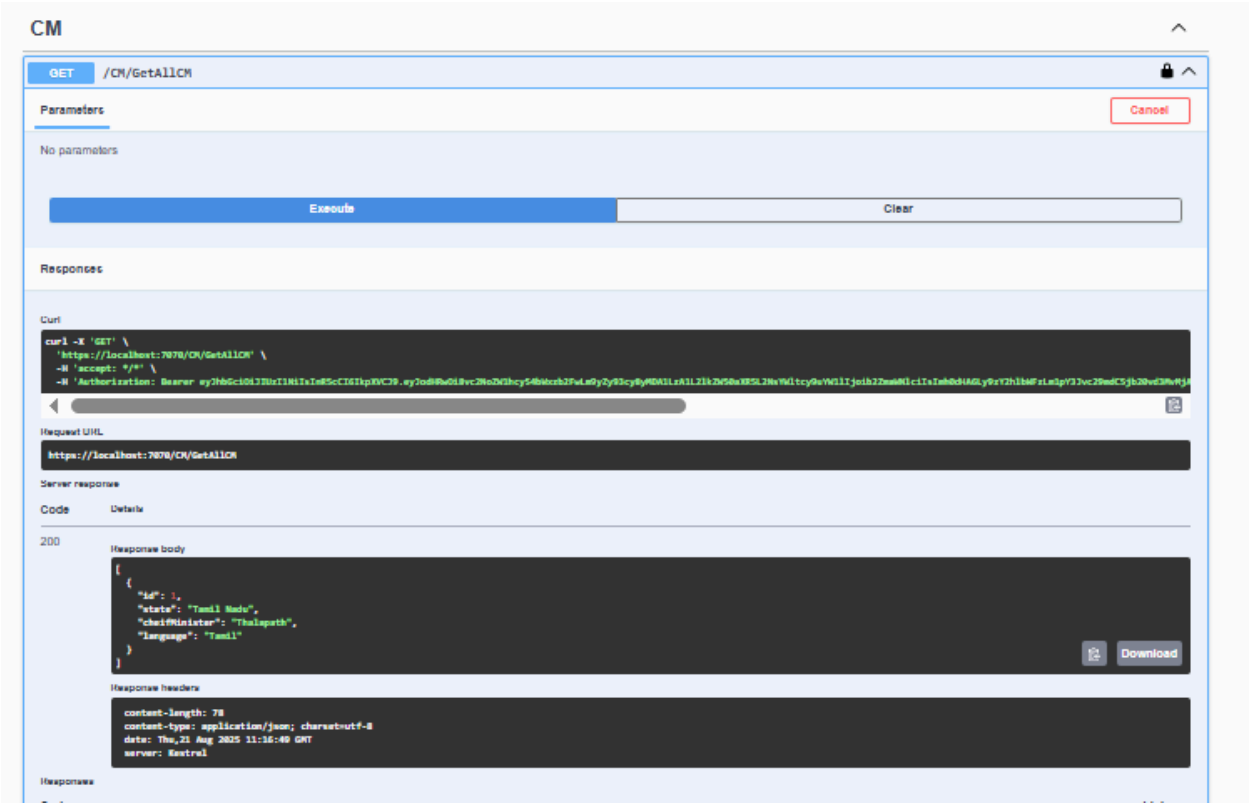
Surya Prakash M
21/8/25

Output:





Surya Prakash M
21/8/25



Controller :

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SampleApi.Context;
using SampleApi.Model;


namespace SampleApi.Controllers
{
    [Route("CM")]
    [ApiController]
    public class CMController : ControllerBase
    {
        private MyContext _context;

        public CMController(MyContext myContext)
        {
            _context = myContext;
        }
```

```
    [Authorize(Roles="officer")]
    [HttpGet("GetAllCM")]
    public IActionResult GetAllCM()
    {
        return Ok(_context.cm.ToList());
    }

    [HttpGet("GetAllCMByID")]
    public IActionResult GetAllCMByID([FromQuery]int ID)
    {
        var cm = _context.cm.FirstOrDefault(m => m.ID == ID);
        if (cm == null)
        {
            NotFound();
        }
        return Ok(cm);
    }

    [Authorize(Roles = "admin")]
    [HttpPost]
    public IActionResult AddCM(CM cm )
    {
        _context.cm.Add(cm);
        _context.SaveChanges();
        return CreatedAtAction(nameof(GetAllCMByID) , new { ID = cm.ID },cm);
    }


  }
}


Model:
  public class CM
  {
    public int ID { get; set; }
    public string State { get; set; }
    public string CheifMinister { get; set; }
    public string Language { get; set; }
  }
```

Surya Prakash M
21/8/25

JwtService:

```
public class JwtService
{
    public static string CreateJWTToken(JWToptions options, IEnumerable<Claim> claims)
    {
        var creds = new SigningCredentials(new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(options.key)),
SecurityAlgorithms.HmacSha256);
        var token = new JwtSecurityToken(
            issuer: options.Issuer,
            audience: options.Audience,
            claims: claims,
            expires: DateTime.Now.AddMinutes(30),
            signingCredentials: creds);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

JwtOptions:

```
public class JWToptions
{
    public string Issuer { get; set; }

    public string Audience { get; set; }
    public string key { get; set; }
    public string ExpireMinutes { get; set; }
}
```

Program.cs:

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using SampleApi.Context;
using SampleApi.Model;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using SampleApi.Controllers;

var builder = WebApplication.CreateBuilder(args);
```

Surya Prakash M
21/8/25

```
builder.Services.AddControllers();

builder.Services.AddControllers().AddNewtonsoftJson();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
   c.SwaggerDoc("v1", new() { Title = "SampleApi", Version = "v1" });

   c.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
   {
      Name = "Authorization",
      Type = Microsoft.OpenApi.Models.SecuritySchemeType.ApiKey,
      Scheme = "Bearer",
      BearerFormat = "JWT",
      In = Microsoft.OpenApi.Models.ParameterLocation.Header,
      Description = "Enter 'Bearer' [space] and then your valid token.\nExample: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6..."
   });

   c.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
   {
      {
         new Microsoft.OpenApi.Models.OpenApiSecurityScheme
         {
            Reference = new Microsoft.OpenApi.Models.OpenApiReference
            {
               Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
               Id = "Bearer"
            }
         },
         new string[] {}
      }
   });
});
builder.Services.AddAutoMapper(typeof(Program));

//string connectionString = "Data Source = PTPLL487;Initial Catalog = Training;Integrated
Security = true;TrustServerCertificate = True;";

builder.Services.Configure<JWToptions>(builder.Configuration.GetSection("Jwt"));
var jwt = builder.Configuration.GetSection("Jwt");
```

```
var issuer = jwt["Issuer"];
var audience = jwt["Audience"];
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt["key"]));

builder.Services.AddAuthentication(options =>
{
   options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
   options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;

}).AddJwtBearer(options =>
{
   options.TokenValidationParameters = new TokenValidationParameters
   {
      ValidIssuer = issuer,
      ValidAudience = audience,
      IssuerSigningKey = key,
      ValidateIssuer = true,
      ValidateAudience = true,
      ValidateIssuerSigningKey = true,
      ValidateLifetime = true,
      ClockSkew = TimeSpan.Zero,
      RoleClaimType = ClaimTypes.Role,
      NameClaimType = ClaimTypes.NameIdentifier
   };

});

string connectionString = builder.Configuration.GetConnectionString("DefaultConnection");


builder.Services.AddDbContext<MyContext>(x => x.UseSqlServer(connectionString));

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
   app.UseSwagger();
   app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapControllers();


app.Run();
```